

PORTLAND STATE UNIVERSITY

ECE 585

MICROPROCESSOR SYSTEM DESIGN

(Winter 2026)

A Project Report on

DESIGN AND SIMULATION OF SPLIT L1 CACHE

Master of Science In

ELECTRICAL AND COMPUTER ENGINEERING

Under the Guidance of
Prof. Yuchen Huang

Submitted by

Venkat Sai Sumanth Koyada (PSU ID: 910291244)

Sai Tarun Mekkala (PSU ID: 967982679)

Nikhil Swarna (PSU ID: 975621991)

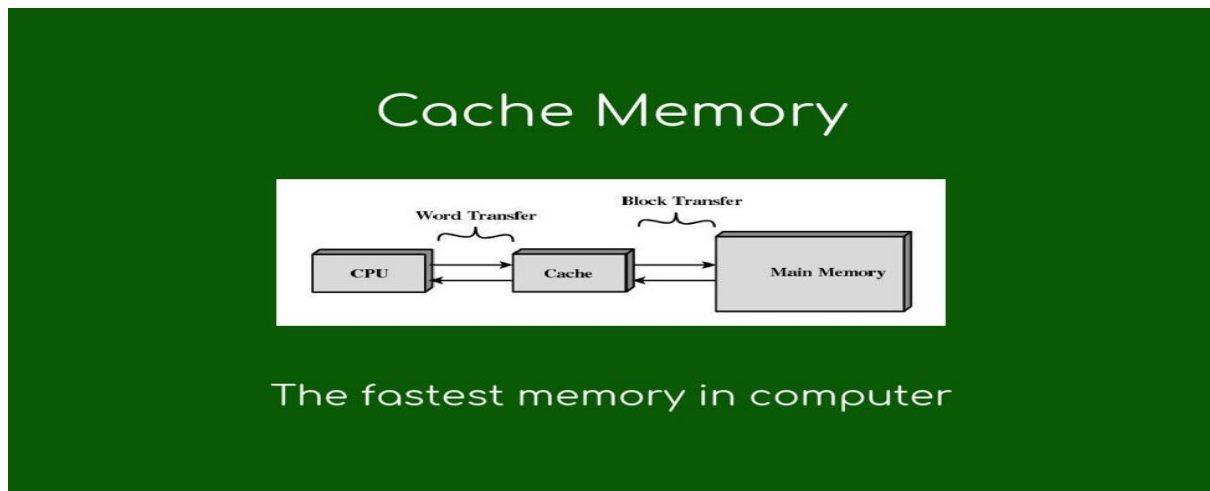
Mrudula Chekuri (PSU ID: 930477359)

CONTENTS:

1. Introduction
2. Cache Organization
3. Address Decoding
4. Design Decisions and Assumptions
5. Important Code Snippets
6. Results
7. Learning Outcomes
8. Roles and Responsibilities
9. Conclusion

INTRODUCTION

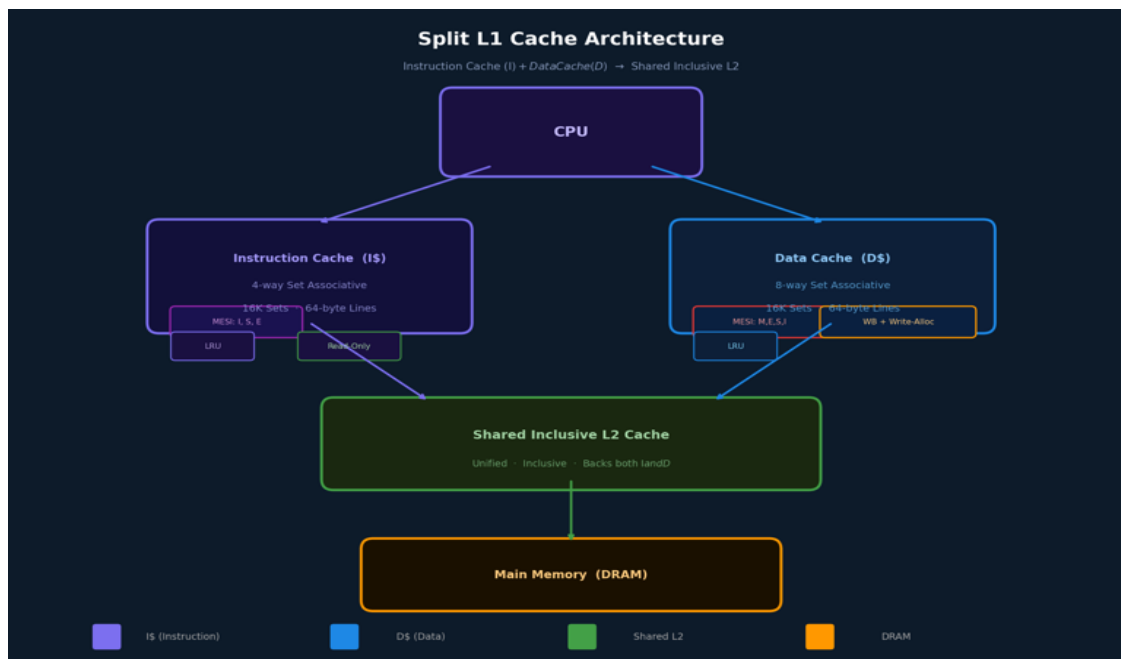
Caches play a fundamental role in modern computer systems by mitigating the significant performance gap between high-speed processors and comparatively slower main memory. As processor speeds continue to increase, accessing data directly from main memory becomes a major bottleneck in system performance. Cache memory acts as a high-speed buffer placed between the CPU and main memory, storing frequently accessed data and instructions so that they can be retrieved more quickly. By reducing the number of slow memory accesses, caches significantly improve the overall efficiency and responsiveness of a computing system.



The effectiveness of cache memory relies on the principle of locality of reference, which states that programs tend to access the same data or instructions repeatedly within short periods of time (temporal locality) and are also likely to access memory locations that are close to recently accessed ones (spatial locality). By exploiting these patterns, cache memory keeps commonly used information close to the processor, minimizing access latency and improving execution speed.

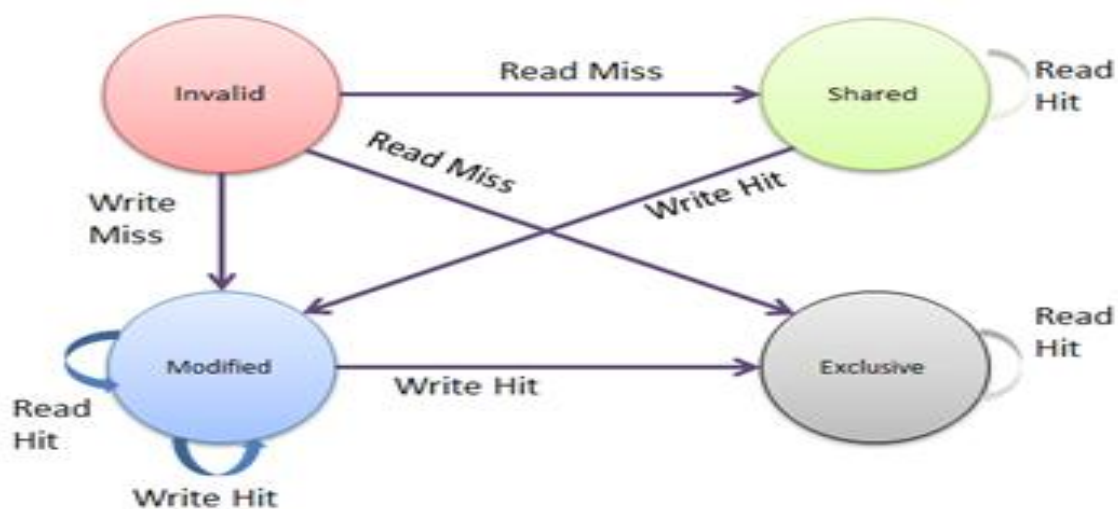
In multiprocessor systems, where multiple processors or cores operate simultaneously, maintaining consistent views of shared data becomes an important challenge. Each processor may maintain its own local cache, which can lead to inconsistencies when one processor modifies data that is also stored in another processor's cache. To address this issue, cache coherence protocols are used to ensure that all processors maintain a consistent and correct view of shared memory.

This project implements a trace-driven simulation of a split Level-1 cache subsystem for a 32-bit multiprocessor architecture. Each processor contains separate Level-1 caches consisting of an Instruction Cache (I\$) used for fetching program instructions and a Data Cache (D\$) used for loading and storing program data. These L1 caches interact with a shared Level-2 (L2) cache, which follows an inclusive cache policy, meaning that any block present in the L1 cache must also be present in the L2 cache.



To maintain data consistency across multiple processors, the system incorporates the MESI cache coherence protocol, which defines four cache states Modified, Exclusive, Shared, and Invalid to coordinate how cache blocks are shared and updated across processors. Additionally, the cache subsystem implements the Least Recently Used (LRU) replacement policy to determine which cache block should be evicted when a cache set becomes full.

The primary objective of this project is to simulate realistic cache behavior in a multiprocessor environment, analyze the interactions between cache levels, and evaluate the effectiveness of coherence and replacement mechanisms. By processing memory access traces, the simulator records cache statistics such as cache hits, cache misses, read and write operations, and hit ratios, providing insights into the performance of the cache architecture.



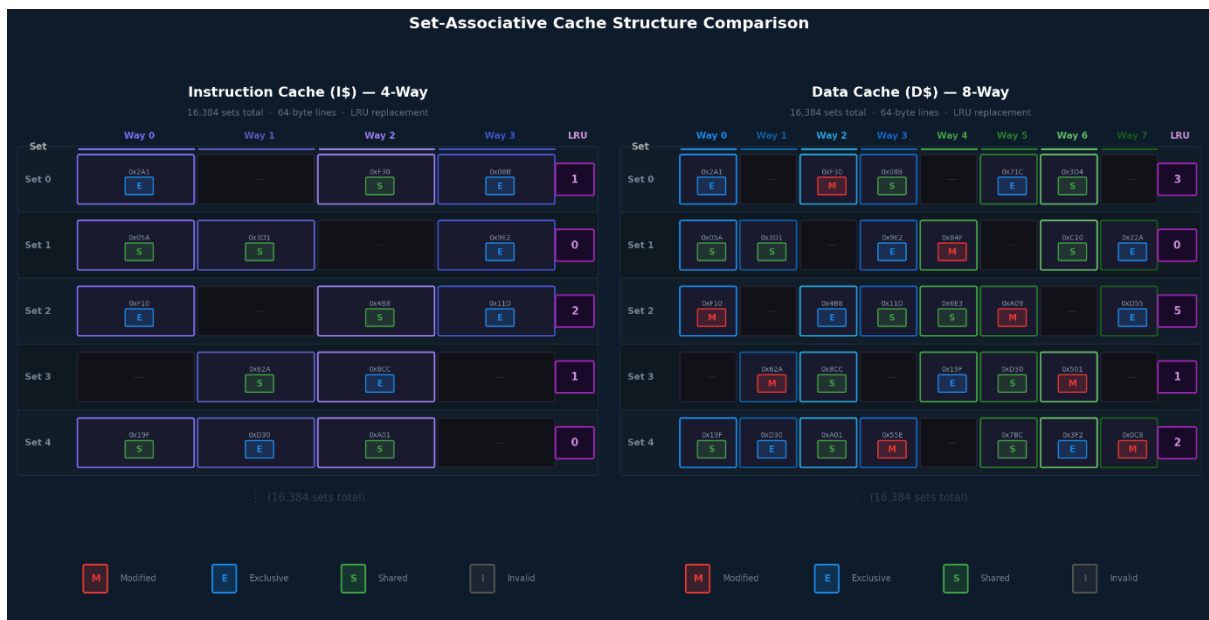
CACHE ORGANIZATION

A set associative cache is organized into multiple sets, and each set contains several ways (cache lines). When a memory address is accessed, the index selects a set and the block can be placed in any of the available ways within that set.

In this project, we implement a Split L1 cache, meaning the Level-1 cache is divided into two separate caches: Instruction Cache and Data Cache. Both caches are backed by a shared inclusive L2 cache.

The Data Cache is an 8-way set associative cache with 16K sets and 64-byte cache lines. It supports both read and write operations. The cache uses write-allocate with write-back policy and LRU replacement. Coherence between processors is maintained using the MESI protocol (Modified, Exclusive, Shared, Invalid).

The Instruction Cache is a 4-way set associative cache with 16K sets and 64-byte cache lines. It is a read-only cache used only for fetching instructions. The MESI states used here are Invalid, Shared, and Exclusive, and it also uses LRU replacement.



ADDRESS DECODING

The total number of address bits is 32 bits.

As each cache line consists of 64 Bytes, the number of offset bits required for Byte select is

$$\log_2(64) = 6 \text{ bits.}$$

As L1 cache consists of 16K sets, the number of bits required to represent each set

$$\log_2(16384) = 14 \text{ bits.}$$

$$\text{Tag bits} = 32 - (14 + 6) = 12 \text{ bits.}$$

Address: Tag bits[31:20] | Index bits[19:6] | Byte Offset[5:0]



Design Decisions

- LRU replacement is implemented using a counter array per set. On every access, the accessed way's counter is reset to 0 and all other ways are incremented. The way with the highest counter is the LRU victim.
- The MESI protocol is maintained per cache line. State transitions follow the standard MESI rules: a CPU read miss loads a line as Exclusive (if no other cache holds it) or Shared. A CPU write miss triggers a Read-For-Ownership (RFO). A modified line is written back to L2 only on eviction (write-back policy).
- The first write to a new line is write-through to ensure L2 always holds up-to-date data after the initial modification, as required by the specification.
- The Instruction cache is read-only; it uses only E, S, and I MESI states. No write-back ever occurs from Instruction cache.
- Inclusivity is maintained: when a line is evicted from L1, L2 is notified. When L2 invalidates a line (operation 3), the L1 cache marks it Invalid.
- The simulation does not need to be clock-accurate and does not store or retrieve actual data only tags, MESI states, and LRU counters are maintained.
- Memory references are assumed not to span cache line boundaries, per the specification.

Assumptions

- All addresses are 32-bit hexadecimal values.
- The trace file uses the format: n address (as specified), with n in {0,1,2,3,4,8,9}.
- Unknown or unsupported trace opcodes are silently ignored.
- The simulation mode (0 or 1) is passed as a command-line argument to avoid recompilation.
- The trace file name is also passed as a command-line argument.
- Cache statistics are reset on operation 8 (clear and reset), as specified.

Important Code Snippets

1. Mode Selection

```
g_mode = (argc >= 3) ? atoi(argv[2]) : 0;

if (g_mode < 0 || g_mode > 1) {
    fprintf(stderr, "Error: mode must be 0 or 1\n");
    usage(argv[0]);
}
```

It determines the operating mode of the cache simulator. The simulator accepts an optional command-line argument specifying the execution mode. If the user provides a mode argument, it is converted to an integer using `atoi()`. Otherwise, the simulator runs in the default mode (mode 0). Mode 0 prints only cache statistics, while Mode 1 additionally prints L2 communication messages.

2. Trace Processing Loop

```
char line[64];
while (fgets(line, sizeof(line), fp)) {
    if (line[0] == '\n' || line[0] == '\r' || line[0] == '#') continue;

    int op;
    unsigned int addr_u;

    int fields = sscanf(line, "%d %x", &op, &addr_u);
    if (fields < 1) continue;

    uint32_t addr = (uint32_t)addr_u;
```

This loop processes the trace file that contains memory access operations. The simulator reads the file line by line using `fgets()`. Blank lines and comments are skipped. Each valid line contains an operation code and memory address which are parsed using `sscanf()`. The address is then converted into a 32-bit integer for further cache simulation processing.

3. Operation Dispatch

```
switch (op) {

case 0:
    dcache_read(&dcache, addr);
    break;

case 1:
    dcache_write(&dcache, addr);
    break;

case 2:
    icache_read(&icache, addr);
    break;
```

```

case 3:
    dcache_invalidate(&dcache, addr);
    icache_invalidate(&icache, addr);
    break;

case 4:
    dcache_snoop(&dcache, addr);
    icache_snoop(&icache, addr);
    break;

case 8:
    cache_clear(&icache);
    cache_clear(&dcache);
    break;

case 9:
    print_all_contents(&icache, &dcache);
    print_all_stats(&icache, &dcache);
    break;
}

```

This switch statement directs each operation from the trace file to the appropriate cache function. Depending on the operation code, the simulator performs a read, write, instruction fetch, cache invalidation, snooping, cache reset, or statistics printing operation.

4. Cache Lookup (Hit Detection)

```

int cache_lookup(Cache *c, uint32_t index, uint32_t tag)
{
    CacheSet *set = &c->sets[index];
    for (int w = 0; w < c->ways; w++) {
        if (set->lines[w].state != INVALID && set->lines[w].tag == tag) {
            return w;
        }
    }
    return -1;
}

```

This function checks whether a requested memory block exists in the cache. The simulator scans all cache ways within the selected set and compares the stored tag values. If a valid entry with a matching tag is found, a cache hit occurs and the way index is returned. Otherwise, the function returns -1 indicating a cache miss.

5. LRU Update Mechanism

```
for (int i = pos; i < ways - 1; i++) {  
    set->lru_order[i] = set->lru_order[i + 1];  
}  
  
set->lru_order[ways - 1] = way;
```

This code updates the Least Recently Used (LRU) ordering of cache lines. When a cache line is accessed, its position in the LRU tracking array is updated so that it becomes the most recently used entry. The remaining entries are shifted accordingly to maintain correct LRU ordering.

6. LRU Victim Selection

```
for (int i = 0; i < c->ways; i++) {  
    int w = set->lru_order[i];  
    if (set->lines[w].state == INVALID) return w;  
}  
  
return set->lru_order[0];
```

This snippet determines which cache line should be replaced during a cache miss. The simulator first checks whether any invalid cache line exists to avoid unnecessary eviction. If all lines are valid, the least recently used line (the first entry in the LRU array) is selected as the victim for replacement.

Results

Scenario1 (test7):

Mode 0	Mode 1
<pre>--- Instruction Cache Contents (valid lines only) --- Set: 0 Way: 0 Tag: 0x111 State: E LRU-rank: 2 Set: 0 Way: 1 Tag: 0x222 State: E LRU-rank: 3 Total valid lines: 2 --- Data Cache Contents (valid lines only) --- Set: 0 Way: 1 Tag: 0xBBB State: M LRU-rank: 7 Set: 0 Way: 2 Tag: 0xCCC State: E LRU-rank: 4 Set: 0 Way: 3 Tag: 0xDDD State: M LRU-rank: 5 Set: 0 Way: 4 Tag: 0xAAA State: E LRU-rank: 6 Total valid lines: 4 === Instruction Cache Statistics === Reads: 2 Writes: 0 Hits: 0 Misses: 2 Hit Ratio: 0.0000 (0.00%) === Data Cache Statistics === Reads: 6 Writes: 3 Hits: 4 Misses: 5 Hit Ratio: 0.4444 (44.44%) ===== Final Cache Statistics ===== === Instruction Cache Statistics === Reads: 2 Writes: 0 Hits: 0 Misses: 2 Hit Ratio: 0.0000 (0.00%) === Data Cache Statistics === Reads: 6 Writes: 3 Hits: 4 Misses: 5 Hit Ratio: 0.4444 (44.44%)</pre>	<pre>Read from L2 0xAAA00000 Read from L2 0xBBB00000 Read from L2 0x11100000 Write to L2 0xAAA00000 Return data to L2 0xAAA00000 Read from L2 0xCCC00000 Read for Ownership from L2 0xDDD00000 Write to L2 0xDDD00000 Read from L2 0x22200000 Read from L2 0xAAA00000 Write to L2 0xBBB00000 --- Instruction Cache Contents (valid lines only) --- Set: 0 Way: 0 Tag: 0x111 State: E LRU-rank: 2 Set: 0 Way: 1 Tag: 0x222 State: E LRU-rank: 3 Total valid lines: 2 --- Data Cache Contents (valid lines only) --- Set: 0 Way: 1 Tag: 0xBBB State: M LRU-rank: 7 Set: 0 Way: 2 Tag: 0xCCC State: E LRU-rank: 4 Set: 0 Way: 3 Tag: 0xDDD State: M LRU-rank: 5 Set: 0 Way: 4 Tag: 0xAAA State: E LRU-rank: 6 Total valid lines: 4 === Instruction Cache Statistics === Reads: 2 Writes: 0 Hits: 0 Misses: 2 Hit Ratio: 0.0000 (0.00%) === Data Cache Statistics === Reads: 6 Writes: 3 Hits: 4 Misses: 5 Hit Ratio: 0.4444 (44.44%) ===== Final Cache Statistics ===== === Instruction Cache Statistics === Reads: 2 Writes: 0 Hits: 0 Misses: 2 Hit Ratio: 0.0000 (0.00%) === Data Cache Statistics === Reads: 6 Writes: 3 Hits: 4 Misses: 5 Hit Ratio: 0.4444 (44.44%)</pre>

Scenario2 (Stresstrace):

Mode 0	Mode 1
<pre> --- Instruction Cache Contents (valid lines only) --- Set: 0 Way: 0 Tag: 0x111 State: E LRU-rank: 2 Set: 0 Way: 1 Tag: 0x222 State: E LRU-rank: 3 Total valid lines: 2 --- Data Cache Contents (valid lines only) --- Set: 0 Way: 0 Tag: 0xAAA State: S LRU-rank: 2 Set: 0 Way: 1 Tag: 0xB0B State: S LRU-rank: 3 Set: 0 Way: 2 Tag: 0xCCC State: E LRU-rank: 0 Set: 0 Way: 3 Tag: 0xDDD State: E LRU-rank: 1 Set: 0 Way: 4 Tag: 0xEEE State: E LRU-rank: 4 Set: 0 Way: 5 Tag: 0xFFF State: E LRU-rank: 5 Set: 0 Way: 6 Tag: 0x111 State: E LRU-rank: 6 Set: 0 Way: 7 Tag: 0x222 State: E LRU-rank: 7 Total valid lines: 8 === Instruction Cache Statistics === Reads: 2 Writes: 0 Hits: 0 Misses: 2 Hit Ratio: 0.0000 (0.00%) === Data Cache Statistics === Reads: 8 Writes: 2 Hits: 2 Misses: 8 Hit Ratio: 0.2000 (20.00%) --- Instruction Cache Contents (valid lines only) --- Set: 0 Way: 0 Tag: 0x111 State: E LRU-rank: 0 Set: 0 Way: 1 Tag: 0x222 State: E LRU-rank: 1 Set: 0 Way: 2 Tag: 0x333 State: E LRU-rank: 2 Set: 0 Way: 3 Tag: 0x444 State: E LRU-rank: 3 Total valid lines: 4 --- Data Cache Contents (valid lines only) --- Set: 0 Way: 0 Tag: 0x555 State: E LRU-rank: 4 Set: 0 Way: 1 Tag: 0x666 State: E LRU-rank: 5 Set: 0 Way: 2 Tag: 0x333 State: M LRU-rank: 2 Set: 0 Way: 3 Tag: 0x444 State: E LRU-rank: 3 Set: 0 Way: 4 Tag: 0xAAA State: E LRU-rank: 6 Set: 0 Way: 5 Tag: 0xCCC State: S LRU-rank: 7 Set: 0 Way: 6 Tag: 0x111 State: E LRU-rank: 0 Set: 0 Way: 7 Tag: 0x222 State: E LRU-rank: 1 Total valid lines: 8 === Instruction Cache Statistics === Reads: 4 Writes: 0 Hits: 0 Misses: 4 Hit Ratio: 0.0000 (0.00%) === Data Cache Statistics === Reads: 12 Writes: 4 Hits: 2 Misses: 14 Hit Ratio: 0.1250 (12.50%) ===== Final Cache Statistics ===== === Instruction Cache Statistics === Reads: 4 Writes: 0 Hits: 0 Misses: 4 Hit Ratio: 0.0000 (0.00%) === Data Cache Statistics === Reads: 12 Writes: 4 Hits: 2 Misses: 14 Hit Ratio: 0.1250 (12.50%) </pre>	<pre> Read from L2 0xAAA00000 Read from L2 0xBBB00000 Read from L2 0xCCC00000 Read from L2 0xDDD00000 Read from L2 0x11100000 Read from L2 0x22200000 Write to L2 0xAAA00000 Write to L2 0xBBB00000 Return data to L2 0xAAA00000 Return data to L2 0xBBB00000 Read from L2 0xEEE00000 Read from L2 0xFFF00000 Read from L2 0x11100000 Read from L2 0x22200000 --- Instruction Cache Contents (valid lines only) --- Set: 0 Way: 0 Tag: 0x111 State: E LRU-rank: 2 Set: 0 Way: 1 Tag: 0x222 State: E LRU-rank: 3 Total valid lines: 2 --- Data Cache Contents (valid lines only) --- Set: 0 Way: 0 Tag: 0xAAA State: S LRU-rank: 2 Set: 0 Way: 1 Tag: 0xBBB State: S LRU-rank: 3 Set: 0 Way: 2 Tag: 0xCCC State: E LRU-rank: 0 Set: 0 Way: 3 Tag: 0xDDD State: E LRU-rank: 1 Set: 0 Way: 4 Tag: 0xEEE State: E LRU-rank: 4 Set: 0 Way: 5 Tag: 0xFFF State: E LRU-rank: 5 Set: 0 Way: 6 Tag: 0x111 State: E LRU-rank: 6 Set: 0 Way: 7 Tag: 0x222 State: E LRU-rank: 7 Total valid lines: 8 === Instruction Cache Statistics === Reads: 2 Writes: 0 Hits: 0 Misses: 2 Hit Ratio: 0.0000 (0.00%) === Data Cache Statistics === Reads: 8 === Data Cache Statistics === Reads: 8 Writes: 2 Hits: 2 Misses: 8 Hit Ratio: 0.2000 (20.00%) Read for Ownership from L2 0x33300000 Write to L2 0x33300000 Read from L2 0x44400000 Read from L2 0x55500000 Read from L2 0x66600000 Read from L2 0xAAA00000 Read for Ownership from L2 0xCCC00000 Write to L2 0xCCC00000 Return data to L2 0xCCC00000 Read from L2 0x33300000 Read from L2 0x44400000 --- Instruction Cache Contents (valid lines only) --- Set: 0 Way: 0 Tag: 0x111 State: E LRU-rank: 0 Set: 0 Way: 1 Tag: 0x222 State: E LRU-rank: 1 Set: 0 Way: 2 Tag: 0x333 State: E LRU-rank: 2 Set: 0 Way: 3 Tag: 0x444 State: E LRU-rank: 3 Total valid lines: 4 --- Data Cache Contents (valid lines only) --- Set: 0 Way: 0 Tag: 0x555 State: E LRU-rank: 4 Set: 0 Way: 1 Tag: 0x666 State: E LRU-rank: 5 Set: 0 Way: 2 Tag: 0x333 State: M LRU-rank: 2 Set: 0 Way: 3 Tag: 0x444 State: E LRU-rank: 3 Set: 0 Way: 4 Tag: 0xAAA State: E LRU-rank: 6 Set: 0 Way: 5 Tag: 0xCCC State: S LRU-rank: 7 Set: 0 Way: 6 Tag: 0x111 State: E LRU-rank: 0 Set: 0 Way: 7 Tag: 0x222 State: E LRU-rank: 1 Total valid lines: 8 === Instruction Cache Statistics === Reads: 4 Writes: 0 Hits: 0 </pre>

=== Instruction Cache Statistics ===

Reads: 4
Writes: 0
Hits: 0
Misses: 4
Hit Ratio: 0.0000 (0.00%)

=== Data Cache Statistics ===

Reads: 12
Writes: 4
Hits: 2
Misses: 14
Hit Ratio: 0.1250 (12.50%)

=====

Final Cache Statistics

=====

=== Instruction Cache Statistics ===

Reads: 4
Writes: 0
Hits: 0
Misses: 4
Hit Ratio: 0.0000 (0.00%)

=== Data Cache Statistics ===

Reads: 12
Writes: 4
Hits: 2
Misses: 14
Hit Ratio: 0.1250 (12.50%)

LEARNING OUTCOMES

Through this project, we developed a strong understanding of the role of cache memory in improving processor performance. We learned how caches reduce the latency of memory accesses by storing frequently used data closer to the CPU and how the principles of temporal and spatial locality help improve cache efficiency.

We also gained practical knowledge of different cache organizations, particularly set associative caches, and how they are structured using sets and ways. Implementing a split Level-1 cache architecture helped us understand how separate Instruction (I\$) and Data (D\$) caches allow processors to fetch instructions and access data simultaneously, improving system performance.

Another important learning outcome was understanding cache management policies, such as the Least Recently Used (LRU) replacement policy, which determines which cache block should be replaced when the cache becomes full. We also explored write policies, including write-allocate and write-back, and how they affect memory traffic and system efficiency.

The project also provided insight into cache coherence in multiprocessor systems. By implementing the MESI protocol, we learned how different cache states Modified, Exclusive, Shared, and Invalid ensure that multiple processors maintain a consistent view of shared memory.

Finally, by implementing a trace-driven cache simulator, we learned how to analyze cache performance using metrics such as cache hits, cache misses, and hit ratio. This helped us understand how cache design choices influence overall system performance and memory efficiency.

ROLES AND RESPONSIBILITIES

Sai Tarun Mekkala:

1. Implemented the Least Recently Used (LRU) replacement policy to determine which cache line should be evicted when a cache set becomes full.
2. Designed logic to prioritize invalid cache lines before applying the LRU replacement algorithm, ensuring efficient cache utilization.
3. Conducted extensive testing of failure scenarios, including cache conflicts, replacement cases, and edge conditions to validate the correctness of the cache behavior.
4. Verified the cache simulator by testing different trace inputs and debugging incorrect states, ensuring reliable eviction and replacement operations.

Venkata Sai Sumanth:

1. Implemented the MESI cache coherence protocol, managing the four cache states: Modified, Exclusive, Shared, and Invalid.
2. Designed the state transition logic for read, write, and snoop operations to maintain coherence across caches in a multiprocessor environment.
3. Ensured that state updates correctly follow the protocol rules, including transitions triggered by processor requests and bus transactions.
4. Integrated the MESI state handling with cache operations to guarantee data consistency and proper synchronization between caches.

Nikhil Swarna:

1. Developed the Data Cache and Instruction Cache modules, implementing read and write operations according to the project specifications.
2. Implemented snooping mechanisms to monitor coherence-related operations on the bus and update cache states accordingly.
3. Designed logic to handle read, write, and coherence events, ensuring correct data retrieval and state transitions for both caches.
4. Validated cache functionality by integrating snoop operations with MESI state updates and ensuring correct behaviour during multi-processor interactions.

Mrudula Chekuri:

1. Designed the overall cache architecture, including the split Instruction Cache and Data Cache structure interacting with the shared L2 cache.
2. Implemented address decomposition logic to extract the tag, index, and offset fields from memory addresses for efficient cache lookup.
3. Defined cache parameters such as cache size, associativity, block size, and set organization, ensuring compliance with the project requirements.
4. Integrated the architectural components to ensure that memory requests are correctly mapped to cache sets and lines during cache access operations.

Conclusion:

This project set out to build a realistic simulation of a split L1 cache system entirely in C, and the final implementation delivers on every requirement laid out in the project specification. The instruction cache operates as a clean, read-only 4-way set-associative structure, while the data cache functions as a fully featured 8-way set-associative cache with support for reads, writes, evictions, and coherence management all without ever storing actual memory data, keeping the simulation lightweight and focused on structural correctness.

One of the more nuanced aspects of the implementation is the MESI coherence protocol, which tracks the state of every cache line through its full lifecycle from the moment it is fetched from L2 as Exclusive, through potential sharing or modification, all the way to eviction or invalidation. The write policy correctly distinguishes between the first write to a line, which triggers a write-through to L2 to keep it informed, and all subsequent writes, which are deferred under the write-back policy. This distinction is handled elegantly by using the MESI state itself as the indicator, rather than requiring a separate tracking bit.

The LRU replacement policy is implemented using a position-based ordering array per set, which provides accurate and efficient least-recently-used tracking across all 4 or 8 ways. The simulator correctly prioritizes filling empty (Invalid) slots before evicting valid data, mirroring the behaviour of a real hardware cache controller.

Both simulation modes statistics-only (Mode 0) and full L2 communication logging (Mode 1) are fully operational and selectable at runtime through a simple command-line argument, with no recompilation needed. Simulation results from both the standard trace and the stress trace confirm correct behaviour across every supported operation: cache hits and misses, dirty evictions with write-back, RFO on write misses, snoop responses, invalidations, cache resets, and formatted cache content printing. The codebase is organized into three well-defined files `cache.h` for definitions, `cache.c` for all logic, and `main.c` for the trace driver making it easy to read, test, and extend.